

# TASK 2 – DEEP LEARNING IMAGE CLASSIFICATION PROJECT

**Submitted By:**

Komaroji Sai Swathika

**Role:**

Artificial Intelligence Intern

**Internship:**

InternSpark AI Internship Program

(AICTE-Listed Internship Program)

---

## 1. INTRODUCTION

Deep Learning is a subset of Artificial Intelligence that uses neural networks to learn complex patterns from data. Image classification is one of the most common applications of deep learning where the model predicts the category of an image.

In this project, a Convolutional Neural Network (CNN) was developed to classify images using the CIFAR-10 dataset.

---

## 2. OBJECTIVE

The main objectives of this project are:

- To understand deep learning concepts
  - To build a CNN model
  - To classify images into categories
  - To evaluate model performance
- 

## 3. DATASET DESCRIPTION

Dataset Name:

CIFAR-10 Dataset

The dataset contains images belonging to ten categories:

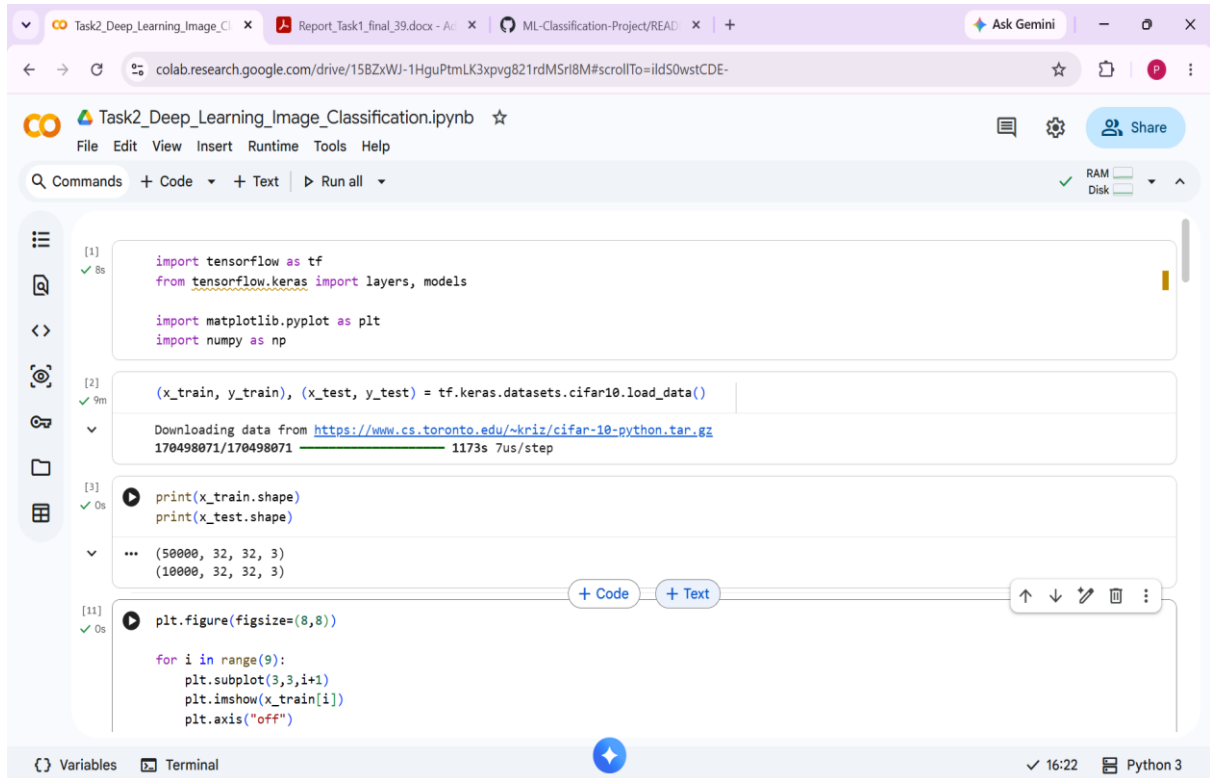
- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse

- Ship
- Truck

Training Images: 50,000

Testing Images: 10,000

Image Size:  
32 × 32 pixels



```
[1] import tensorflow as tf
    from tensorflow.keras import layers, models

    import matplotlib.pyplot as plt
    import numpy as np

[2] (x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 1173s 7us/step

[3] print(x_train.shape)
    print(x_test.shape)

... (50000, 32, 32, 3)
    (10000, 32, 32, 3)

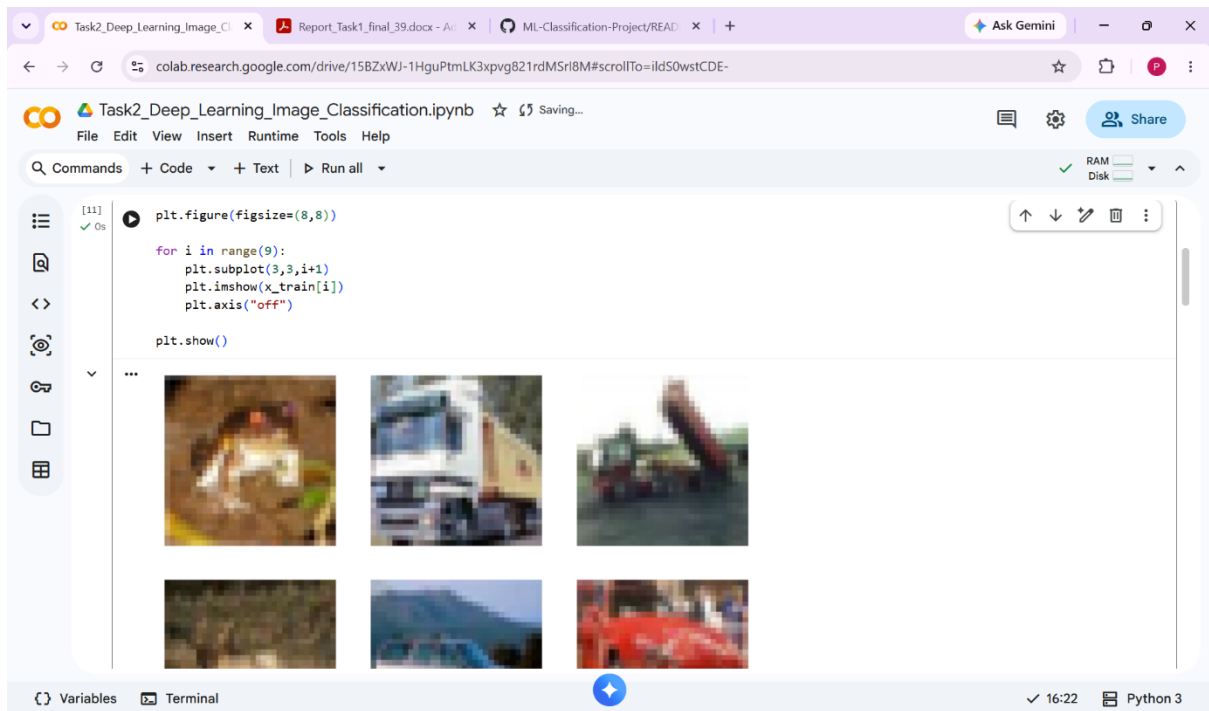
[11] plt.figure(figsize=(8,8))

    for i in range(9):
        plt.subplot(3,3,i+1)
        plt.imshow(x_train[i])
        plt.axis("off")
```

#### 4. DATA PREPROCESSING

Data preprocessing techniques performed:

- Loaded dataset
- Normalized image values
- Converted image pixels between 0–1
- Split training and testing data



## 5. MODEL ARCHITECTURE

The CNN model consists of:

Input Layer



Convolution Layer



Max Pooling Layer



Convolution Layer



Max Pooling Layer



Flatten Layer



Dense Layer



Output Layer

## 6. MODEL TRAINING

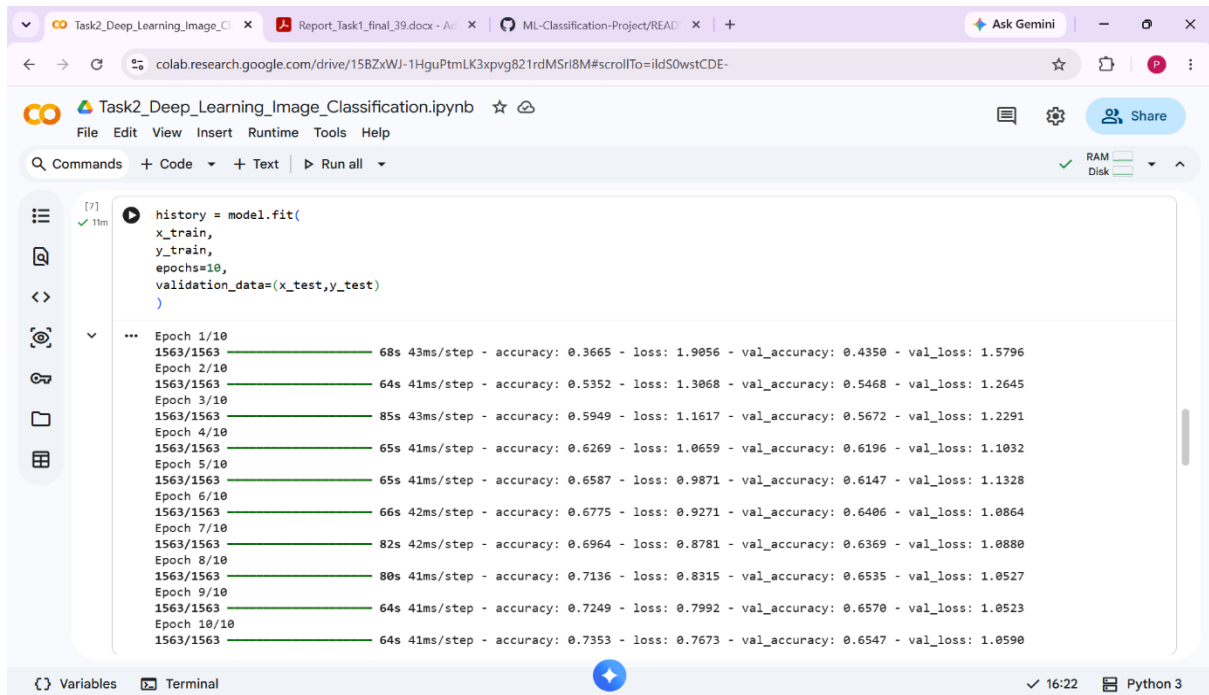
Model configuration:

Optimizer: Adam

Loss Function: Sparse Categorical Crossentropy

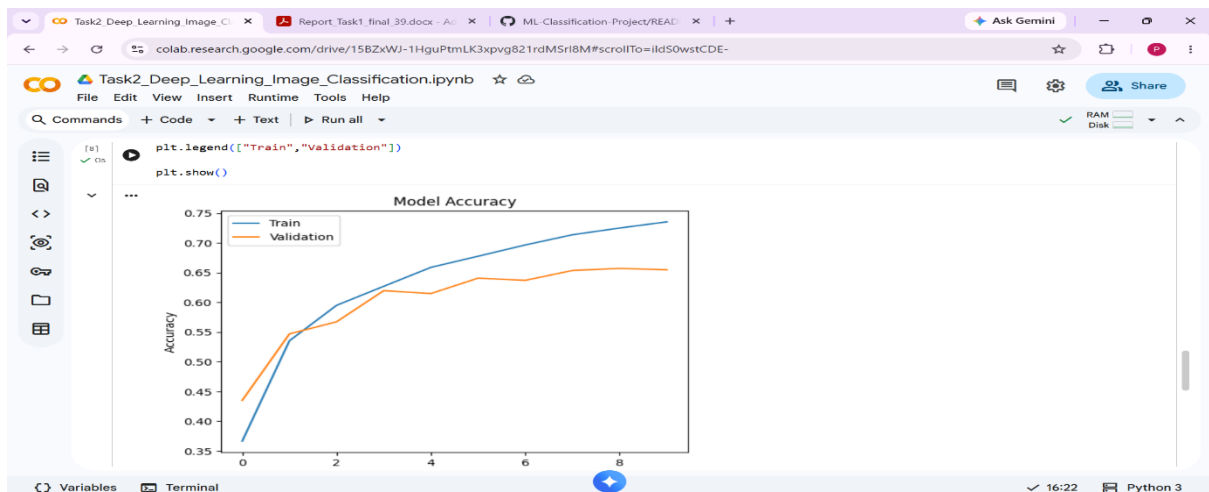
Epochs:10

Evaluation Metric: Accuracy



## 7. TRAINING CURVE

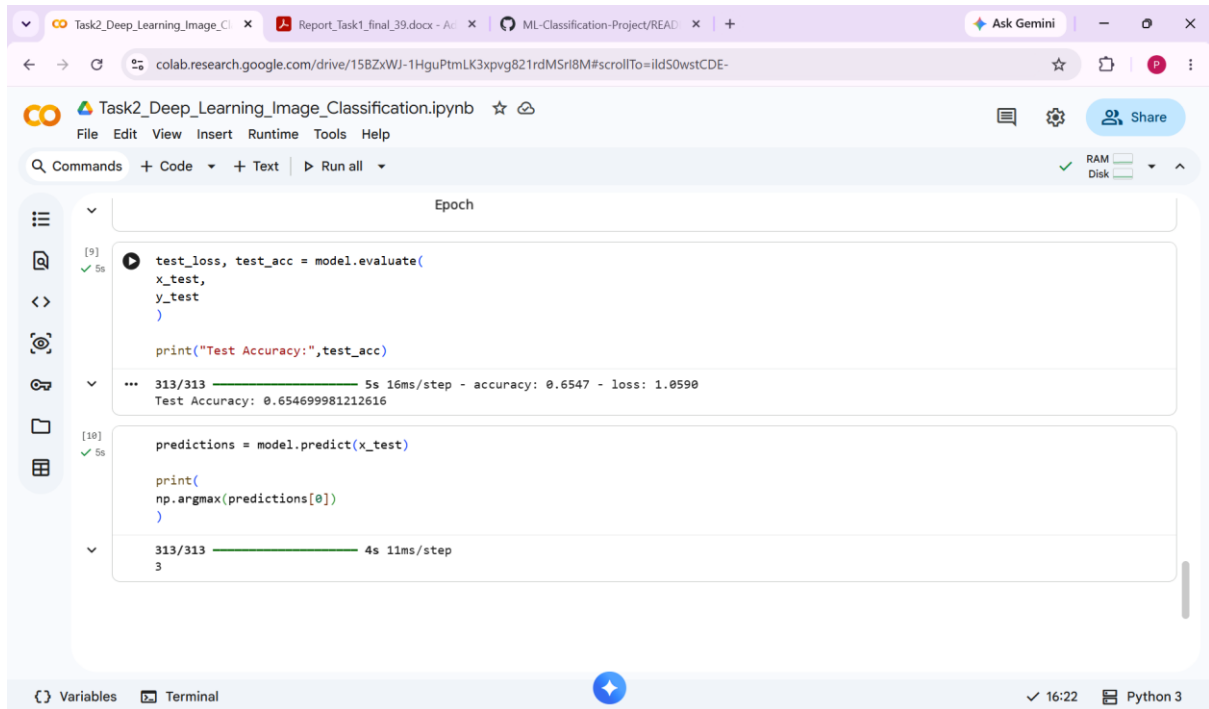
Training and validation accuracy curves were plotted to observe the model performance during training.



---

## 8. MODEL EVALUATION

The trained model was evaluated using testing data.



The screenshot shows a Google Colab notebook titled "Task2\_Deep\_Learning\_Image\_Classification.ipynb". The notebook is open to a cell labeled "Epoch" which contains two code blocks. The first code block (cell [9]) evaluates the model using `model.evaluate(x_test, y_test)` and prints the test accuracy. The output shows a progress bar for 313/313 steps, with a test accuracy of 0.654699981212616. The second code block (cell [10]) uses `model.predict(x_test)` to generate predictions and prints the first prediction using `np.argmax(predictions[0])`. The output shows a progress bar for 313/313 steps and the prediction value 3. The notebook interface includes a left sidebar with icons for file explorer, search, and other tools. The bottom status bar shows "Variables", "Terminal", a blue star icon, "16:22", and "Python 3".

```
[9] test_loss, test_acc = model.evaluate(x_test, y_test)
    print("Test Accuracy:", test_acc)

... 313/313 5s 16ms/step - accuracy: 0.6547 - loss: 1.0590
Test Accuracy: 0.654699981212616

[10] predictions = model.predict(x_test)
     print(np.argmax(predictions[0]))

... 313/313 4s 11ms/step
3
```

---

## 9. CONCLUSION

The CNN model successfully classified images into different categories using deep learning techniques. The model achieved good prediction accuracy and demonstrated the effectiveness of CNN architecture in image classification tasks.

---